Problem Statement #43 — Feature Flag & Dynamic Config Manager

Requirements Engineering & UML Use-Case Modelling

Developer Tools & IT Operations | Actors: Software Engineer · Release Manager | 5 FR · 2 NFR · 1 Use-Case Flow

---

1. Complete Requirements Table

ID	Type	Description	Priority	Acceptance Criteria	Rationale
FR-001	**Functional**	**Feature Flag Evaluation with Targeting Rules** The system shall evaluate user targeting rules (e.g., User ID hash, Beta cohort membership) and return an accurate boolean feature flag state per request.	**High**	✔ **Pass:** User within the 10 % rollout cohort receives `enabled = true`. ✘ **Fail:** User outside the cohort receives `enabled = true`.	Core system behaviour; incorrect evaluation directly impacts feature delivery and A/B integrity.

ID	Type	Description	Priority	Acceptance Criteria	Rationale
FR-002	**Functional**	**Percentage-Based Cohort Rollout Configuration** The system shall allow a Release Manager to configure a percentage-based rollout (0–100 %) per feature flag, deterministically assigning user IDs to enabled or disabled cohorts via consistent hashing.	**High**	✔ **Pass:** Setting rollout to 25 % results in exactly 25 ± 1 % of a 10,000-user test set receiving `enabled = true`. ✘ **Fail:** Distribution deviates beyond ± 2 % across repeated evaluations for the same user set.	Enables safe, gradual rollouts; consistent hashing ensures the same user always gets the same state regardless of evaluation order.
FR-003	**Functional**	**Environment-Specific Flag Override** The system shall support per-environment flag overrides (development, staging, production), allowing a Release Manager to force-enable or force-disable a flag in a specific environment independently of global rollout configuration.	**High**	✔ **Pass:** A flag force-enabled in *staging* returns `true` for all users in that environment while returning rollout-based state in *production*. ✘ **Fail:** Override in one environment bleeds into another.	Prevents staging test flags from leaking to production; enables QA and canary testing in isolation.

ID	Type	Description	Priority	Acceptance Criteria	Rationale
FR-004	**Functional**	**Real-Time Client SDK Flag Synchronization** The system shall push flag state changes to all connected client SDKs within 2 seconds of a configuration update, using a server-sent events (SSE) or WebSocket streaming channel.	**High**	✔ **Pass:** Flag state change is reflected in SDK-cached state within 2 s under 1,000 concurrent SDK connections. ✘ **Fail:** SDK continues serving stale flag state after 5 s.	Eliminates stale evaluation windows; critical for kill-switch scenarios where an instant flag disable is required.
FR-005	**Functional**	**Audit Log and Change History** The system shall maintain an immutable, timestamped audit log of all flag creation, update, override, and deletion events, capturing the actor (user/API key), previous state, and new state.	**Medium**	✔ **Pass:** Every configuration change produces a queryable audit entry within 1 s, retained for ≥ 90 days. ✘ **Fail:** A log entry is missing, mutable, or lacks actor attribution.	Supports compliance, incident root-cause analysis, and rollback decisions by providing full configuration lineage.

ID	Type	Description	Priority	Acceptance Criteria	Rationale
NFR-001	**Performance**	**Low-Latency Flag Evaluation API** The feature flag evaluation API endpoint shall complete evaluation and return a response in under 15 ms at p99 under simulated peak load, to minimise overhead on client critical paths.	**High**	✔ **Pass:** Load test (500 RPS, 60 s) confirms p99 latency ≤ 15 ms with zero evaluation errors. ✘ **Fail:** p99 exceeds 15 ms or error rate exceeds 0.1 % under load.	Flag evaluation is called synchronously in application hot paths; any latency adds directly to end-user response time across all served requests.
NFR-002	**Availability & Reliability**	**High Availability with Graceful Degradation** The system shall maintain 99.9 % uptime (≤ 8.7 h downtime/year) and, in the event of a control-plane outage, SDK clients shall fall back to their last-cached flag states rather than failing open or closed unpredictably.	**High**	✔ **Pass:** Simulated control-plane failure results in SDK serving cached flag state within 500 ms without throwing exceptions. ✘ **Fail:** SDK throws errors or returns undefined state during outage, or uptime SLA is breached over a 30-day window.	Feature flags often gate critical functionality; an unavailable flag service must not cause cascading application failures.

2. Use-Case Flow Specification

Evaluate Feature Flag *UC-003 | Core Use Case*

Primary Actor: Software Engineer (via Client SDK / App)

Supporting Actor: Release Manager (has configured flags & rules)

Trigger: Application code calls `flagClient.evaluate(flag_key, user_ctx)`

Related FR: FR-001, FR-002, FR-003, FR-004, FR-005

Preconditions

- The client application has been authenticated with a valid SDK API key.
- The feature flag identified by `flag_key` exists in the system and is in an *active* state.
- A user context object containing `user_id` and relevant attributes is available in the evaluation request.
- The flag's targeting rules, rollout percentage, and environment overrides have been configured by a Release Manager.

Postconditions (on Success)

- A boolean flag state (`true` / `false`) is returned to the client SDK within 15 ms.
- An evaluation event is appended to the audit log with timestamp, `user_id`, flag key, returned state, and matched rule.
- If the returned flag state differs from the SDK's locally cached value, the SDK cache is updated and a `flagChanged` event is emitted.

Main Success Scenario

1. The Software Engineer's application invokes the SDK evaluation method with payload `{flag_key, user_id, user_attributes}` to the Flag Evaluation API over HTTPS.
2. The system validates the SDK API key against the authorised key registry and verifies the request payload schema.
3. The system identifies the target environment (e.g., *production*) from the API key context and retrieves the corresponding flag configuration record.
4. The system checks if an environment-specific override is active for this flag. If found, it returns the override state immediately (skip to Step 9).
5. The system iterates through the flag's ordered targeting rules, evaluating each rule's conditions against the supplied `user_attributes` (e.g., `cohort = "beta"`, `region = "IN"`).
6. A targeting rule matches. The system resolves the flag state defined by the matching rule and proceeds to Step 9.

7. No targeting rule matches. The system computes the user's cohort bucket by deterministically hashing `user_id + flag_key` to an integer in [0, 100).
8. The cohort bucket is compared against the configured rollout percentage. If bucket < rollout %, the flag state is `true`; otherwise `false`.
9. The system returns the evaluated boolean flag state to the client SDK. Total response time is ≤ 15 ms.
10. The system asynchronously writes an evaluation audit entry (timestamp, `user_id`, `flag_key`, resolved state, matched rule / rollout bucket) to the audit log store.

Alternate Flow A — User Outside Rollout Cohort (at Step 8)

⚡ ***Trigger: At Step 8 — the computed cohort bucket ≥ rollout percentage threshold.***

The system resolves the flag state as `false` (feature disabled for this user).

The system returns `{ flag_key, enabled: false, reason: "PERCENTAGE_ROLLOUT_EXCLUDED" }` to the SDK.

The SDK caches the disabled state locally; the application continues serving the fallback / control experience.

Resume at Step 10 — an audit log entry is still written with state `false` and reason `PERCENTAGE_ROLLOUT_EXCLUDED`.